

AMCL Adaptive Monte Carlo Positioning

1. AMCL Positioning

Positioning involves determining the robot's position in the global map. While SLAM also includes positioning algorithm implementation, SLAM's positioning is utilized for building the global map and occurs before navigation begins. Current positioning, on the other hand, is employed during navigation, where the robot needs to move according to the designated route. Through positioning, it can be assessed whether the robot's actual trajectory aligns with expectations.

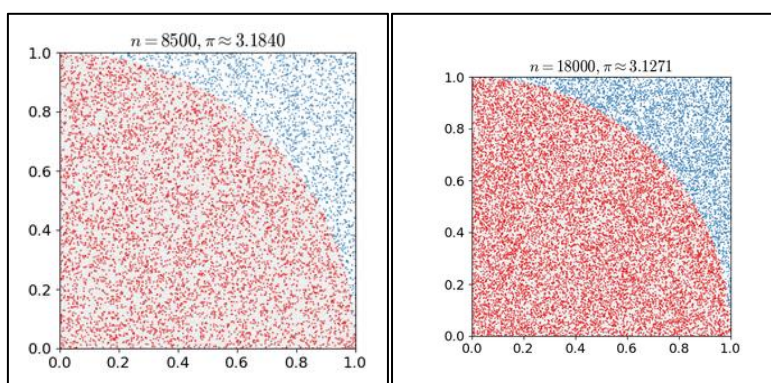
The AMCL (Adaptive Monte Carlo Localization) system, provided in the ROS navigation function package `ros-navigation`, facilitates robot positioning during navigation. AMCL utilizes the adaptive Monte Carlo positioning method and employs particle filters to compute the robot's position based on existing maps. Positioning addresses the association between the robot and obstacles, as path planning fundamentally involves decision-making based on surrounding obstacles. While theoretically, completing the navigation task only requires knowledge of the robot's global positioning and real-time obstacle avoidance using laser radar and other sensors, the real-time performance and accuracy of global positioning are typically limited. Local positioning from odometry, IMUs, etc., ensures the real-time performance and accuracy of the robot's motion trajectory. The `amcl` node in the navigation function package provides global positioning by publishing `map_odom`. However, users can substitute `amcl` global positioning with other methods that provide `map_odom`, such as SLAM, UWB, QR code positioning, etc.

Global positioning and local positioning establish a dynamic set of tf coordinates `map_odombase_footprint`, with static tf coordinates between various sensors in the robot provided through the robot URDF type. This TF

relationship resolves the association problem between the robot and obstacles. For instance, if the lidar detects an obstacle 3m ahead, the tf coordinates between the lidar and the robot chassis, base_link to laser_link, are utilized. Through standard transformation, the relationship between the obstacle and the robot chassis can be determined.

2. Particle Filter

The Monte Carlo positioning process simulates particle updates for a one-dimensional robot. Initially, a group of particles is randomly generated, with each particle representing a potential position, direction, or state variable that requires estimation. Each particle is assigned a weight indicating its similarity to the actual system state. Next, the state of each particle at the next time step is predicted, and particles are moved based on the anticipated behavior of the real system. The weights of the particles are then updated based on measurements. Particles that closely match the measured value receive higher weights and are resampled, while highly unlikely particles are discarded and replaced with more probable ones. Finally, the weighted average and covariance of the particle set are calculated to derive the state estimate.



Monte Carlo methods generally follow a specific pattern:

1. Define possible input fields.
2. Randomly generate inputs from a probability distribution over the domain.

3. Perform deterministic calculations on inputs.
4. Summarize results.

Two important considerations are:

1. If the points are not uniformly distributed, the approximation effect will be poor.
2. This process requires many points. The approximation is usually poor if only a few points are randomly placed throughout the square. On average, the accuracy of the approximation improves as more points are placed.

The Monte Carlo particle filter algorithm finds applications in various fields, including physical science, engineering, climatology, and computational biology.

3. Adaptive Monte Carlo Positioning

AMCL can be viewed as an enhanced iteration of the Monte Carlo positioning algorithm, designed to boost real-time performance by employing a reduced number of samples compared to the traditional method, thereby minimizing execution time. It implements an adaptive or KLD sampling Monte Carlo localization method, utilizing particle filtering to track a robot's pose against an existing map.

The Adaptive Monte Carlo positioning nodes primarily utilize laser scanning and lidar maps to exchange messages and perform pose estimation calculations. The implementation process entails initializing the adaptive Monte Carlo positioning algorithm's particle filter for each parameter provided by the ROS system during initialization. In instances where the initial pose is not specified, the algorithm assumes that the robot commences its journey from the origin of the coordinate system, resulting in a more intricate calculation process.

Hence, it is advisable to set the initial pose using the "2D Pose Estimate" button in rviz. For further information on Adaptive Monte Carlo positioning, you

can also refer to the wiki address link:

<https://github.com/ros-planning/navigation>

AMCL can be viewed as an enhanced iteration of the Monte Carlo positioning algorithm, designed to boost real-time performance by employing a reduced number of samples compared to the traditional method, thereby minimizing execution time. It implements an adaptive or KLD sampling Monte Carlo localization method, utilizing particle filtering to track a robot's pose against an existing map.

The Adaptive Monte Carlo positioning nodes primarily utilize laser scanning and lidar maps to exchange messages and perform pose estimation calculations. The implementation process entails initializing the adaptive Monte Carlo positioning algorithm's particle filter for each parameter provided by the ROS system during initialization. In instances where the initial pose is not specified, the algorithm assumes that the robot commences its journey from the origin of the coordinate system, resulting in a more intricate calculation process.

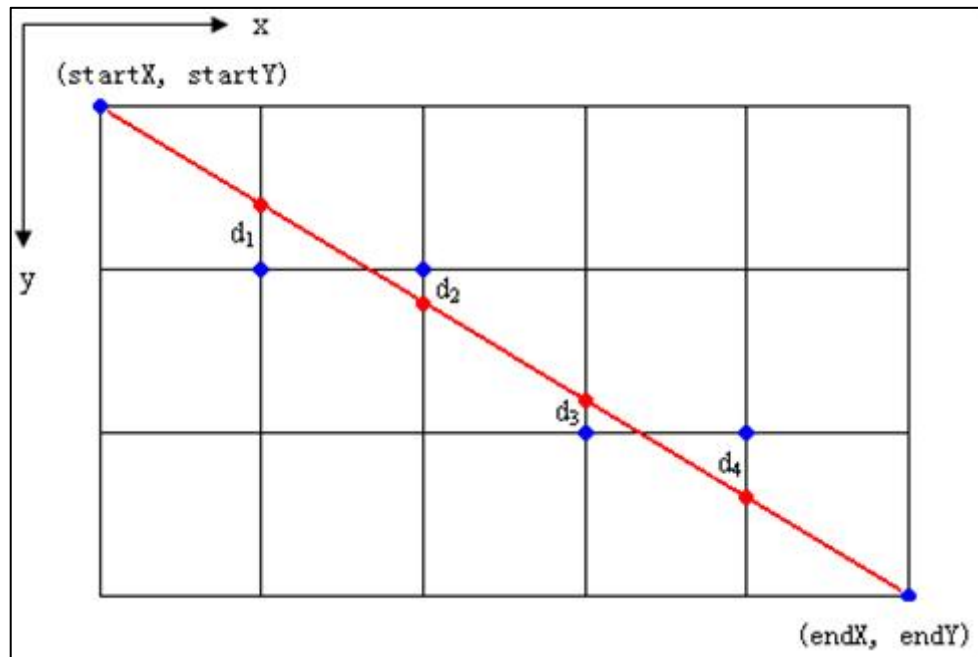
Hence, it is advisable to set the initial pose using the "2D Pose Estimate" button in rviz. For further information on Adaptive Monte Carlo positioning, you can also refer to the wiki address link:

<https://github.com/ros-planning/navigation>

4. Cost Map

Regardless of whether it's a 2D or 3D SLAM map generated by LiDAR or a depth camera, it cannot be directly employed for actual navigation. Instead, it needs to be converted into a costmap. In ROS, the costmap typically adopts a grid format, where each grid cell in the raster map occupies 1 byte (8 bits). This allows for data storage ranging from 0 to 255, representing different cell costs. The costmap only requires consideration of three scenarios: occupied (barrier), free area (barrier-free), and unknown space.

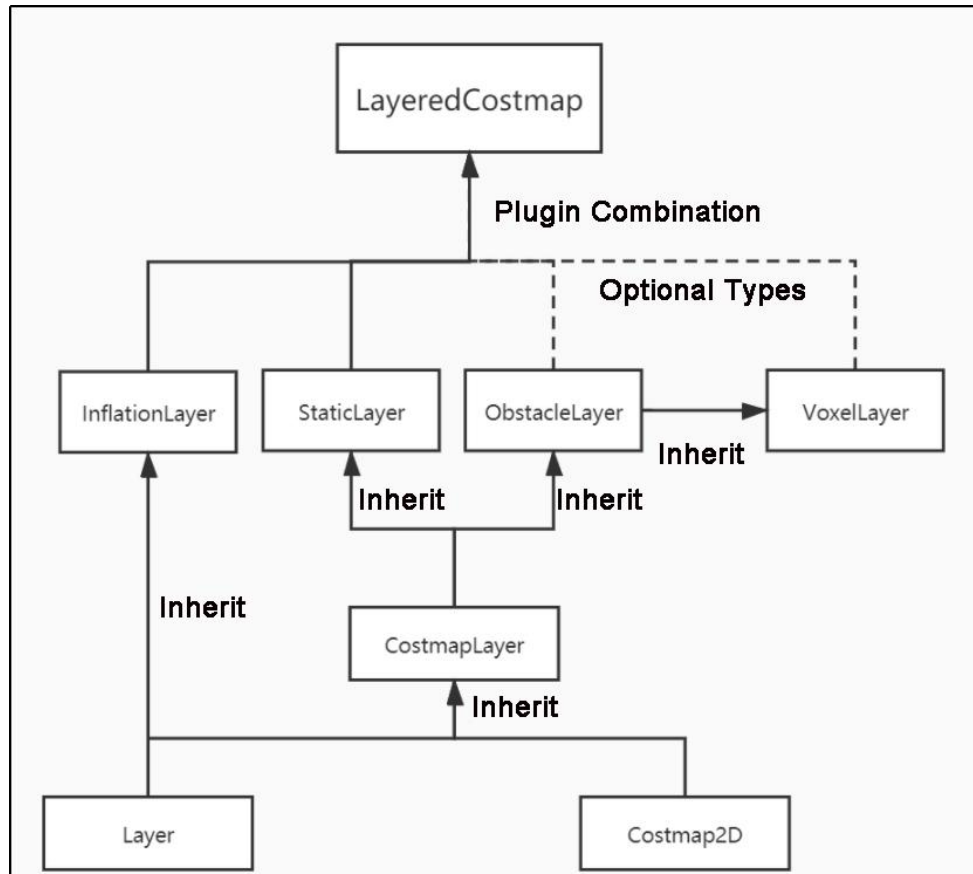
Before delving into `costmap_2d`, it's important to introduce the Bresenham algorithm. This algorithm is utilized to draw a straight line between two points by calculating the closest point of a line segment on an n-dimensional raster. It relies solely on relatively fast integer operations such as addition, subtraction, and bit shifting, making it a fundamental algorithm in computer graphics.



To construct a set of virtual grid lines, we begin by passing them through the pixel centers of each row and column. The intersection points of these straight lines with each vertical grid line are calculated sequentially, starting from the line's origin and moving towards its endpoint. Subsequently, the pixel closest to each intersection point within the pixel column is determined based on the sign of the error term.

The core concept of the algorithm relies on the assumption that $k=dy/dx$, where k represents the slope. Since the straight line originates from the center of a pixel, the initial error term d is set to $d_0=0$. With each increment in the subscript X , d increases by the slope value k , i.e., $d=d+k$. When $d \geq 1$, 1 is subtracted from it to ensure that d remains between 0 and 1. If $d \geq 0.5$, the pixel closest to the upper-right of the current pixel is considered (i.e., $(x+1, y+1)$); otherwise, it's closer to the pixel on the right (i.e., $(x+1, y)$). For ease of

calculation, let $e = d - 0.5$; initially, e is set to -0.5 , and it increments by k . When $e \geq 0$, the upper-right pixel of the current pixel (x_i, y_i) is selected, and when $e < 0$, the pixel closer to the right ($x+1, y$) is chosen instead. Integers are preferred to avoid division. Since the algorithm only uses the sign of the error term, it can be replaced as follows: $e1 = 2 * e * dx$.



The **Costmap2D** class is responsible for managing the cost value of each raster. Meanwhile, the **Layer** class serves as a virtual base class, standardizing the interfaces of costmap layers for each plugin. Key interface functions include:

The **initialize** function, which invokes the **onInitialize** function to initialize each costmap layer individually.

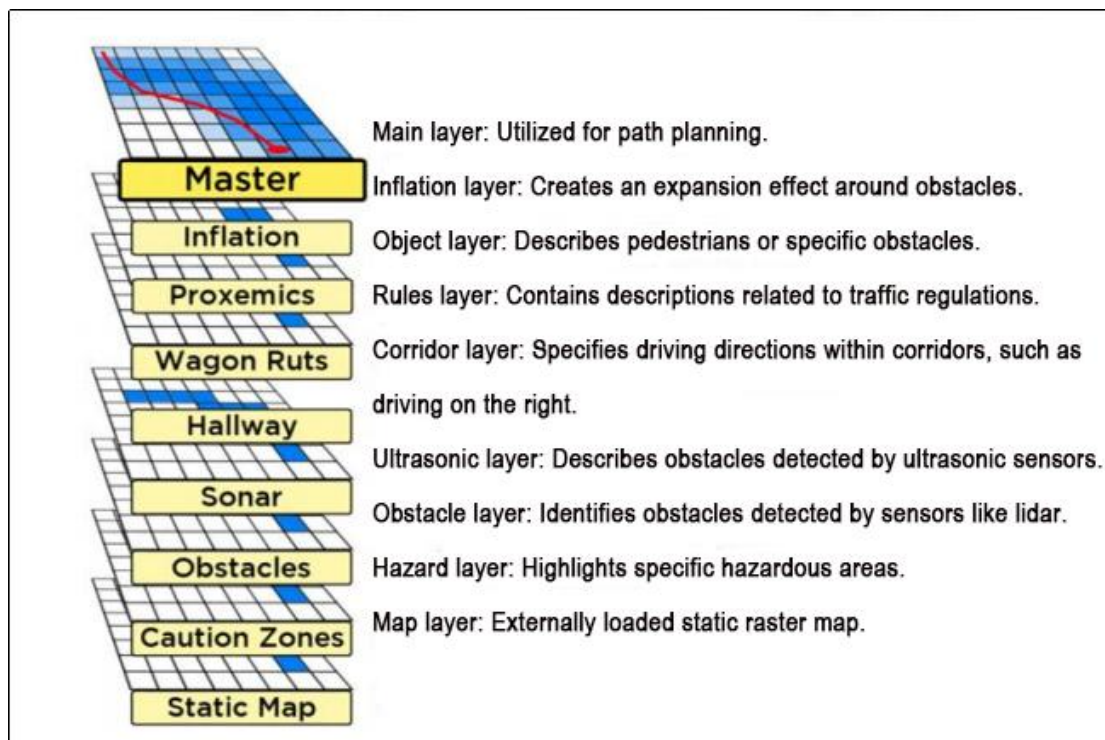
The **matchSize** function, found in the **StaticLayer** and **ObstacleLayer** classes, ensures consistency across costmap layers by calling the **matchSize** function of the **CostmapLayer** class. This initialization process sets the size, resolution,

origin, and default cost value of each layer. In the `inflationLayer` class, a cost table is computed based on the expansion radius, enabling subsequent cost value queries based on distance. Additionally, the `seen_` array is defined to mark traversed rasters. For the `VoxelLayer` class, initialization involves setting the size of the voxel grid.

The `updateBounds` function adjusts the size range of the current costmap layer requiring updates. For the `StaticLayer` class, the update range is determined by the size of the static map (typically used in global costmaps). Conversely, the `ObstacleLayer` class determines obstacle boundaries by traversing sensor data in `clearing_observations`.

The `initialize` and `matchSize` functions are executed only once each, while `updateBounds` and `updateCosts` are performed periodically, their frequency determined by `map_update_frequency`.

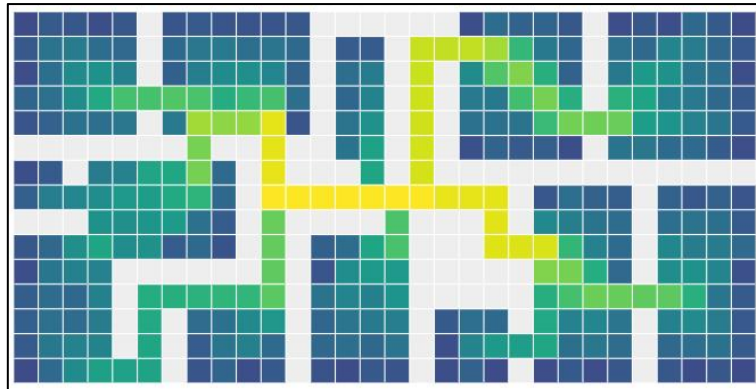
The `CostmapLayer` class inherits from both the `Layer` class and the `Costmap2D` class, providing various methods for updating cost values. The `StaticLayer` and `ObstacleLayer` classes, needing to retain cost values of instantiated layers, also inherit from `CostmapLayer`. The `StaticLayer` updates its costmap using static raster map data, whereas the `ObstacleLayer` employs sensor data for updates. In contrast, the `VoxelLayer` class considers z-axis data more extensively, particularly in obstacle clearance. This distinction primarily affects obstacle removal, with two-dimensional clearance in one case and three-dimensional clearance in the other.



Costmap measurement barriers offer remarkable flexibility. You can tailor a specific layer to your requirements, thereby managing pertinent barrier information effectively. For instance, if your robot is equipped solely with LiDAR, you'll need to establish an Obstacles layer to handle obstacle data scanned by the LiDAR. In case ultrasonic sensors are integrated into the robot, creating a new Sonar layer becomes necessary to manage obstacle information from the sonic sensor. Each layer can define its own rules for obstacle updates, encompassing tasks such as obstacle addition, deletion, and confidence level updates. This approach significantly enhances the scalability of the navigation system.

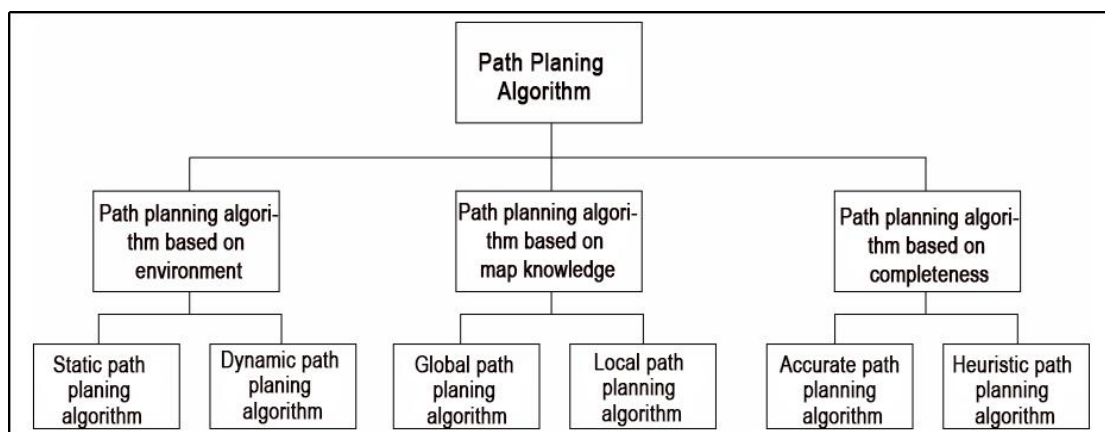
5. Global Path Planning

Preface: Based on the mobile robot's perception of the environment, the characteristics of the environment, and the algorithms employed, path planning can be categorized into environment-based, map knowledge-based, and completeness-based path planning algorithms.



Commonly used path planning algorithms in robot autonomous navigation encompass Dijkstra, A*, D*, PRM, RRT, genetic algorithms, ant colony algorithms, fuzzy algorithms, and others.

Dijkstra and A* are graph-based path search algorithms commonly utilized in robotic applications. The navigation function package integrates navfn, global planner, and carrot planner as global route planning plug-ins. Users have the option to select one of these plug-ins to load into move_base for utilization. Alternatively, they can opt for a third-party global path planning plug-in, such as SBPL_Lattice_Planner or srl_global_planner, or develop a custom global path planning plug-in adhering to the interface specification of nav_core.



Mobile robot navigation facilitates reaching a target point through path planning. The navigation planning layer comprises several components:

1) **Global path planning layer:** This layer generates a global weight map based on the provided goal and accepts weight map information. It then plans a global path from the starting point to the target location, serving as a

reference for local path planning.

2) **Local path planning layer:** This component, constituting the local planning aspect of the navigation system, operates on the local weight map information derived from the weight map. It conducts local path planning considering nearby obstacle information.

3) **Behavior execution layer:** This layer integrates instructions and path planning data from the higher layers to determine the current behavior of the mobile robot.

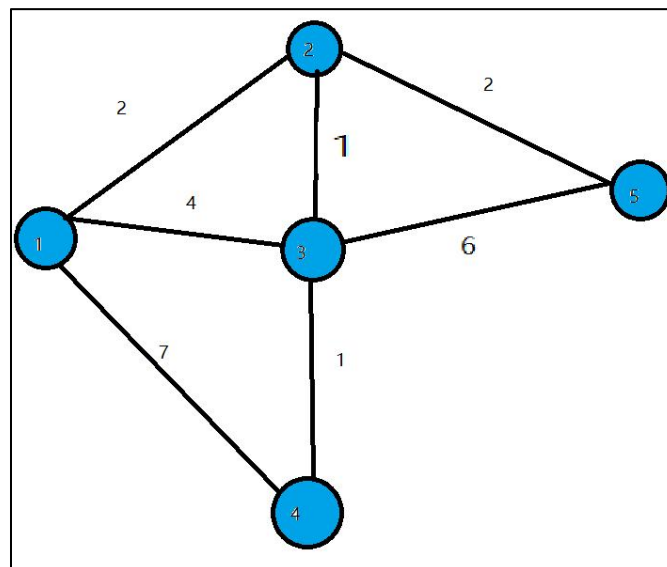
Path planning algorithms for mobile robots are a crucial area of research, significantly impacting the efficiency of robotic operations.

5.1 Dijkstra Algorithm

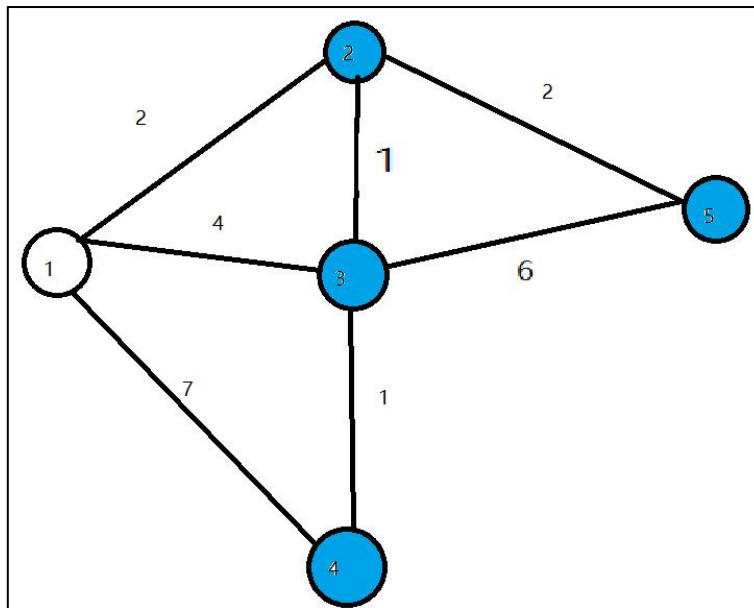
Dijkstra's algorithm is a classic shortest path algorithm known for its efficiency in finding the shortest path from a single source to all other vertices in a graph. It operates by expanding outward in layers from the starting point, employing a breadth-first search approach while considering edge weights. This makes it one of the most widely used algorithms in global path planning.

Here is a diagram illustrating the process of Dijkstra's algorithm:

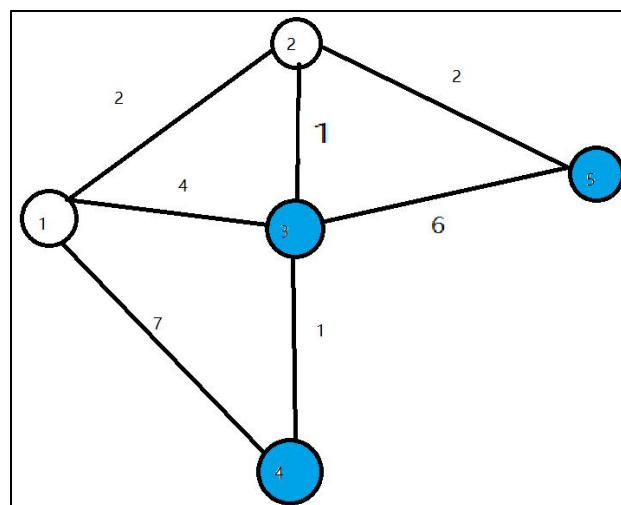
1. Initially, we set the distance from the starting point (start) to itself as 0, and all other points' distances are initialized to infinity.



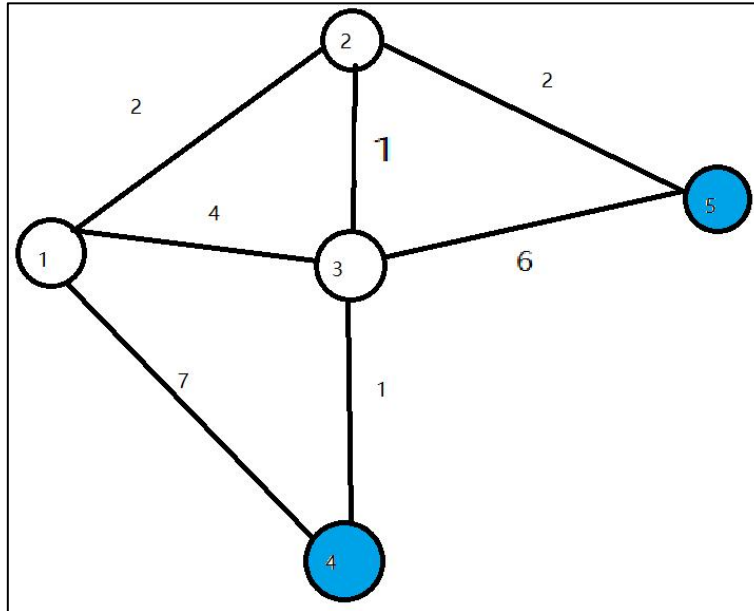
2. During the first iteration, we identify the point (let's call it Point 1) with the smallest distance value, mark it as processed, and update the distances of all adjacent points (previously unprocessed) connected to Point 1. For instance, if Point 1 is connected to Points 2, 3, and 4, we update their distances as follows: $\text{dis}[2]=2$, $\text{dis}[3]=4$, and $\text{dis}[4]=7$.



3. In the subsequent iterations, we repeat the process: find the point with the smallest distance value (e.g., Point 2 in the second iteration), mark it as processed, and update the distances of its adjacent points (if necessary). For example, if Point 2 is connected to Points 3 and 5, we update their distances as $\text{dis}[3]=3$ and $\text{dis}[5]=4$.



4. This procedure continues until all points have been processed. At each iteration, we select the unprocessed point with the smallest distance and update distances of its adjacent points accordingly.



5. Once all points have been processed, the algorithm terminates, and the shortest path distances from the starting point to all other points are determined.

To access the introduction and usage details of the Dijkstra algorithm, please log in to the wiki using the following link:

<http://wiki.ros.org/navfn>

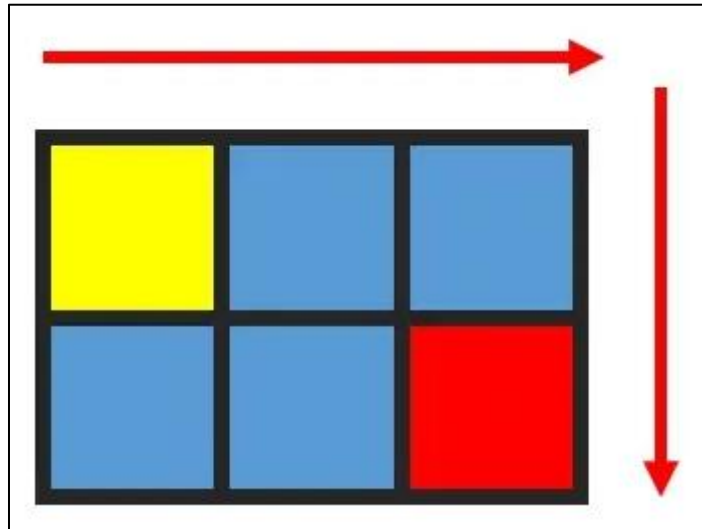
5.2 A Star Algorithm

A-star is an enhancement of Dijkstra's algorithm tailored for single destination optimization. While Dijkstra's algorithm determines paths to all locations, A-star focuses on finding the path to a specific location or the nearest location among several options. It prioritizes paths that seem to be closer to the goal.

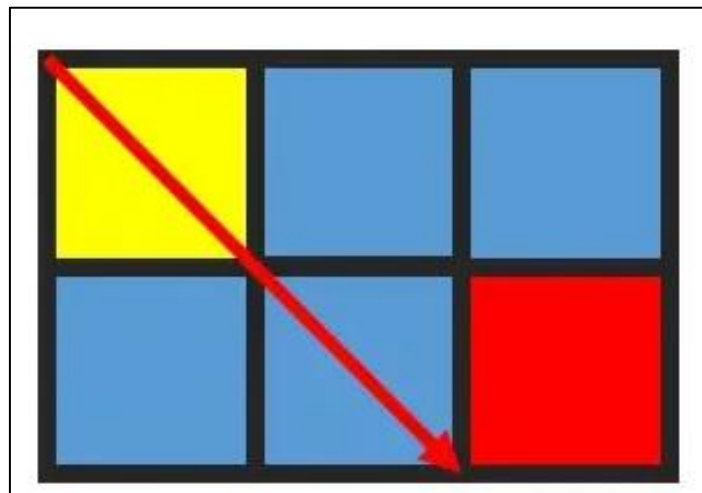
The formula for the A-star algorithm is: $F=G+H$, where G represents the movement cost from the starting point to the designated square, and H denotes the estimated cost from the designated square to the endpoint. There are two methods for calculating the H value:

1. Calculate the distance of horizontal and vertical movement; diagonal

calculation is not applicable (Manhattan distance).



2. Calculate the distance of horizontal and vertical movement; diagonal calculation is applicable (diagonal distance).



For an introduction to and usage of the A* algorithm, please consult the video tutorial or visit the following links:

ROS Wiki: http://wiki.ros.org/global_planner

Red Blob Games website:

<https://www.redblobgames.com/pathfinding/a-star/introduction.html#graphs>